Skill Cartridges: Composable, Hot-Swappable Deletion Masks for Small Language Models

Austin Dixson
Independent researcher
austintdixson@gmail.com

Preprint, November 11, 2026

Abstract

Deletion Tuning [18] adapts a pretrained language model by learning a binary mask that sets a small fraction of its MLP gate-projection weights to zero, and nothing else. We treat these masks as *skill cartridges*: small files that are applied to, and removed from, one frozen base model at request time. Using the authors' method and released code as our base, we test how far the idea goes on single-GPU budgets. A ToolAlpaca cartridge for Qwen3.5-9B raises exact-match accuracy from 42 to 69.3 ± 1.5 of 97 items across three training seeds (each seed $p \leq 3 \times 10^{-8}$, exact McNemar test). The same recipe on a 27B model reaches 65. Masks from different seeds share 12 to 13 times more deleted weights than chance would predict, and 777,898 weights are deleted by all three seeds, where 489 would be expected. On a 4-bit MLX build of the 9B running on an Apple-silicon Mac, all three seeds keep the gain: from a shared base of 38 of 97 (identical per item across repeats), the nearest-zero form scores 66, 66 and 61, and requantize scores 66, 65 and 62, with at most one item lost per seed. Swapping in a prepared cartridge takes 0.04 ms, and ejecting it restores the base weights bit for bit. The union of two published Qwen2.5-7B masks keeps 85% of the tool-use gain and 97.9% of the science gain. The same union operator on two cartridges we trained for Qwen3.5-9B keeps 87.5% and 90.8%. The method also works on LiquidAI's LFM3-1.7B, a hybrid convolution and attention model (11 to 59 of 97). A cartridge trained on synthetic latest-value recall lifts the 9B from 89 to 200 of 200 and from 8 to 100 of 100 on a longer held-out variant. A code cartridge trained on MBPP reference solutions made the model worse, and we report it in full. The cartridge imitates those short, unguarded solutions, which are weaker than what the base model already writes, so the result reflects the training targets and does not show a limit of deletion itself. On a 4-bit 27B build, four general-knowledge tasks drop by a mean of 0.76 points on a 34% subset, and no single task drops by more than 1.2 points. One 9B training run costs about $0.60.[1]

1 Introduction

Lu et al. [18] showed that a large language model can learn new tasks when post-training is restricted to deleting pretrained weights. Their method, Deletion Tuning, learns one logit per gate-projection weight, anneals a sigmoid of those logits into a hard keep/delete decision, and deploys the result as a one-bit mask. Across GRPO, supervised fine-tuning and on-policy self-distillation it matches full fine-tuning at the largest scale they test, the masks are hundreds to thousands of times smaller than a fine-tuned checkpoint, and the model forgets less. They also point out that the masks are

[1] Code: https://github.com/austindixson/skill-cartridges. Cartridges: https://huggingface.co/austindixson/skill-cartridges.

cheap to switch, and in their limitations section they suggest that masks for different tasks "might also be combined, for example by taking their union."

This paper takes those observations as a starting point and asks what is needed to run a small local model as one frozen set of weights plus a shelf of masks, one per skill, chosen per request. We call each mask a *skill cartridge*. The term is ours and describes a deployment pattern; the training method is Deletion Tuning as published. Whether the pattern works depends on questions the original paper did not set out to answer: whether a small model with a cartridge can stand in for a larger one, whether the masks are reproducible across seeds, whether they survive the 4-bit quantization that small machines need, whether two cartridges can be active at once, and whether the method carries over to other architectures and skills.

What comes from prior work. The training objective, the sigmoid relaxation and sharpness schedule, the Wanda-based initialization, the sparse-weight regularizer, the mask file format and the ToolAlpaca and Science evaluation settings are those of Lu et al. [18], who in turn build on MaskLLM [9], Wanda [26] and the SDFT task setup of Shenfeld et al. [25]. The two Qwen2.5-7B masks used in our stacking test are the authors' published masks. Our training script is a modified copy of their `train_sft.py` at a pinned commit (Appendix A).

What is new here. All of the following were measured by us on November 10–11, 2026:

1. **A smaller model with a cartridge matches a larger one with a cartridge.** A Qwen3.5-9B ToolAlpaca cartridge scores 69.3 ± 1.5 of 97 against 65 for a cartridge trained with the same recipe on a 27B model, and against 42 for both base models (§5.1).
2. **Replication across seeds and a mask-overlap analysis.** All three 9B seeds are individually significant, and their deleted sets overlap far above chance, with a shared core of 777,898 weights (§5.2).
3. **4-bit retention with a live toggle on Apple silicon.** Two ways of applying a mask to an affine 4-bit model keep the gain on all three 9B ToolAlpaca seeds. We report apply and eject timings and verify exact restoration (§5.3).
4. **Union stacking.** We test the union operator that Lu et al. proposed as future work, on their two published 7B masks and on two cartridges we trained on the 9B (§5.4).
5. **A second architecture.** A cartridge on LiquidAI LFM3-1.7B, whose layers mix gated short convolutions with attention (§5.5).
6. **A recall cartridge.** A cartridge for latest-value key–value recall that reaches ceiling and transfers to a longer held-out format (§5.6).
7. **A negative result for code.** An MBPP-trained cartridge lowered MBPP and HumanEval accuracy (§5.7).
8. **Forgetting, perplexity and KL on the 27B cartridge.** Four off-task benchmarks drop by a mean of 0.76 points on a 34% subset; WikiText-2 perplexity does not rise (§5.8).
9. **Low cost.** Every cartridge in this paper was trained on one rented GPU; a 9B run takes about 13 minutes and $0.60 (§5.9).

2 Background: Deletion Tuning

We summarize the method of [18]. For a target weight matrix $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ (here every MLP gate projection) a logit matrix G of the same shape is trained while all pretrained weights stay frozen. During training the layer uses

$$\widetilde{W}(t) = W \odot \sigma(\alpha(t)\, G), \qquad \alpha(t) = \alpha_{\text{start}} + \tfrac{t}{T}(\alpha_{\text{end}} - \alpha_{\text{start}}), \tag{1}$$

and at deployment the hard mask $\widehat{W} = W \odot \mathbf{1}[G > 0]$. Logits are initialized from Wanda scores $s_{ij} = |W_{ij}| \, \|X_j\|_2$ [26]: weights in the bottom η quantile receive logits drawn near -0.03 and the rest near $+0.03$. The SFT loss is the token-level negative log-likelihood of reference answers plus a MaskLLM-style regularizer $-\lambda\|\widetilde{W}\|_2^2$ [9]. No sparsity target is set. The deployed mask is bit-packed and zstd-compressed.

3 Skill cartridges

A *cartridge* is a Deletion Tuning hard mask $M \in \{0,1\}^N$ over the N gate-projection weights of a specific base checkpoint, stored in the upstream format (per layer: shape and zstd-compressed `np.packbits` of the keep mask, bit 1 = keep). We use $D(M)$ for its deleted set.

Composition. Because a cartridge can only delete, two cartridges compose by set operations on deleted sets. The *union* $D(M_1) \cup D(M_2)$ deletes every weight either cartridge deletes, and is the natural "both skills on" operator. The *intersection* $D(M_1) \cap D(M_2)$ serves as a control. Composition is order-independent and needs no training.

Applying a cartridge to a 4-bit model. Affine group quantization as implemented in MLX [13] stores, per group of 64 input columns of a row, a scale s and bias b, and per weight a 4-bit code q, with dequantized value $sq + b$. Zero is generally not representable. We compare two forms:

- **Nearest-zero.** Keep s, b and all kept codes; replace each deleted weight's code with the code $q^\star = \arg\min_q |sq + b|$ under MLX's own dequantization. No kept weight changes. This form can be applied live to an already quantized model.
- **Requantize.** Dequantize the layer, zero the deleted weights, and quantize again. Deleted weights become exactly zero more often, but scales, biases and many kept codes change.

On the Qwen3.5-9B 4-bit build with the seed-1 mask (10,702,823 deleted weights), nearest-zero lands 7,459,823 deleted weights exactly on 0 and changes no kept code; requantize lands 6,344,744 exactly on 0 and changes the dequantized value of 239,560,390 kept weights, because bf16 dequantize-then-quantize is not idempotent.

Live toggle. The loader keeps a reference to the original arrays of every touched gate projection. *Cold apply* computes the new arrays from the mask and swaps them in. *Warm apply* swaps in arrays computed earlier. *Eject* swaps the originals back. Every timed region ends with `mx.eval`.

4 Experimental setup

Models. Qwen/Qwen3.5-9B (bf16, 32 layers, gate projection 12288×4096, $N = 1{,}610{,}612{,}736$); mlx-community/Qwen3.5-9B-MLX-4bit (affine, 4 bits, group size 64, converted from the same checkpoint; revision `938d8919`); a Qwen3.8-27B checkpoint, for which we trained our own masks; Qwen2.5-7B-Instruct [23] with the two published masks; and LiquidAI/LFM3-1.7B [16] (24 layers: 13 gated short convolutions, 5 sliding-window attention, 6 global attention), where the mask targets `feed_forward.w1`, the gate of its gated MLP ($24 \times 7680 \times 2048 = 377{,}487{,}360$ weights). Qwen3.5-9B and the 27B model are different Qwen generations.

Tasks.

- **ToolAlpaca** [27], in the version prepared by Shenfeld et al. [25] (4,046 training examples, 97 test items), scored by exact match with their `eval_tooluse.py` at commit `d7757321`.

- **Science**: the SciKnowEval Chemistry L-3 subset [10] as prepared by [25] (507 test items), scored with their `eval_science.py`.
- **Latest-value recall** (ours). A prompt lists `name: value` assignment lines for 16 variables, each assigned a uniformly random 4 to 20 times in shuffled order (values 100 to 999), and asks for the current value of 4 of them. An item counts as correct only if all 4 values are right. Training uses 4,000 generated items; the 200 test items come from a fixed separate seed. A *longer held-out* set (100 items) uses 32 variables assigned 20 to 40 times each, so it has twice the variables and roughly five times as many lines (about 7.6k versus 1.6k tokens). This format was never seen in training.
- **Code**: training on the MBPP train, validation and prompt splits (474 problems, reference solutions as targets) [3]; evaluation by pass@1 under greedy decoding on the 500 MBPP test problems and on the 164 HumanEval problems [5], with a 10 s execution timeout.

Training. Every cartridge we trained used the same supervised fine-tuning (SFT) recipe, which follows the upstream `scripts/train_sft.sh` defaults: 400 optimizer steps, batch size 4, AdamW at learning rate 5×10^{-4}, no weight decay, gradient clipping at 1.0, $\eta = 1/2048$, Wanda prior strength 3, $(\alpha_{\text{start}}, \alpha_{\text{end}}) = (100, 1000)$ and $\lambda = 10^{-5}$ decayed linearly to 0. Wanda calibration uses 256 training prompts. The loss is mean cross-entropy over answer tokens only. Maximum sequence length is 2048 (3072 for recall). Thinking mode is off. Training ran on one NVIDIA RTX PRO 6000 (96 GB) through Hugging Face Jobs, except the 27B model, which used two. Our deviations from upstream `train_sft.py` (building the mask once per optimizer step with the regularizer gradient added analytically, applying the LM head only at answer positions, fused AdamW, layer splitting over devices) change memory use and speed, not the objective; a unit test checks that the regularizer gradient matches. The two published Qwen2.5-7B masks (ToolAlpaca and Science) were not trained by us: Lu et al. trained them with on-policy self-distillation (SDFT, upstream `scripts/train_sdft.sh`), so comparisons between them and our cartridges also differ in training recipe.

Evaluation. Greedy decoding with the model's chat template, at most 1024 new tokens for ToolAlpaca, 2048 for Science and 64 for recall. The 9B ToolAlpaca seed scores, and the LFM, recall and code numbers in bf16, come from each training job's own evaluation in PyTorch. The 9B Science score and the 9B stacking numbers were produced with MLX 0.32.4 and mlx-lm 0.32.2 on an Apple-silicon Mac (bf16); under that harness the 9B ToolAlpaca base scores 45 of 97, three above the PyTorch figure of 42. The 27B bf16 ToolAlpaca numbers come from the training job. The 4-bit ToolAlpaca numbers, the 7B stacking numbers, and the 27B forgetting, perplexity and KL numbers were produced with MLX 0.32.4 and mlx-lm 0.32.2 on Apple-silicon Macs. On the first eight ToolAlpaca items the MLX harness reproduced the PyTorch reference responses for the bf16 9B base exactly.

Statistics. Base and cartridge are scored on the same items, so we report items gained and lost and an exact two-sided McNemar test [21] (binomial test on the discordant pairs). For mask overlap we report Jaccard $J = |A \cap B|/|A \cup B|$ against the chance value $J_0 = pq/(p+q-pq)$ for independent deletions at densities p and q, and observed intersections against the chance expectation Npq (or $Npqr$ for three masks).

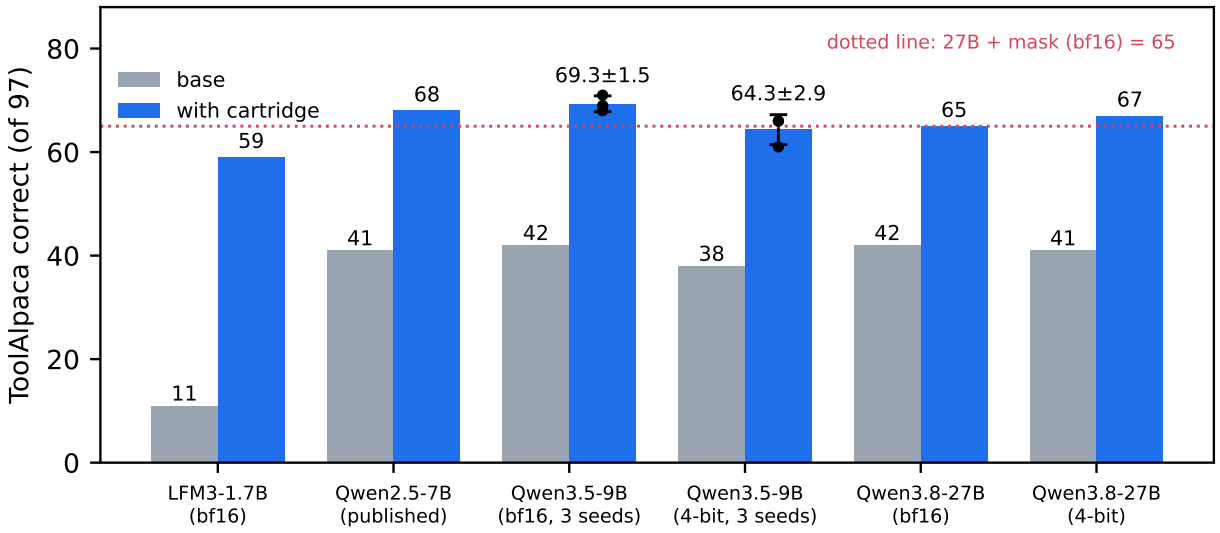


Figure 1: ToolAlpaca exact match (of 97) with and without a cartridge. The 9B bf16 and 4-bit bars are means of three seeds (dots), with ±1 s.d. The 7B mask is the published one from [18], trained with SDFT; all others were trained by us with SFT. 4-bit 9B numbers use the nearest-zero form.

5 Results

5.1 A 9B cartridge matches a 27B cartridge

Table 1 and Figure 1 give the results. The 9B cartridge lifts the model from 42 to a three-seed mean of 69.33 ± 1.53 items ($71.5 \pm 1.6\%$). The 27B model with a cartridge trained by the same recipe scores 65; the 27B base scores 42, the same as the 9B base. The 9B with a cartridge is therefore at least as good as the 27B with one on this benchmark, with about a third of the parameters. We do not claim the 9B is significantly *better*: the 4.3-item difference is between different models on 97 items and we have not run a paired test. The two models are also from different Qwen generations, so model generation and size are confounded.

The 9B gains are almost one-directional: across three seeds, 83 items flip from wrong to right and 1 flips from right to wrong. Each seed's 95% interval on the gain lies between 18.6 and 39.2 percentage points.

5.2 Seeds agree, and the masks share a core

The three seeds delete 10,702,823, 10,950,222 and 10,819,819 weights. Each pair shares about 0.9 million, 12 to 13 times the chance expectation (Table 2, Figure 2). Most of that is common to all three: 777,898 weights, or 86% of a typical pairwise intersection, are deleted by every seed, against 489 expected by chance. The union of the three deleted sets is 30,528,058. So each run deletes a large seed-specific set plus a reproducible core of about 0.05% of gate-projection weights. Per-layer deletion rates follow the same profile across seeds (Figure 3), apart from seed 2's higher rate in layers 0 to 4.

Two cautions apply. First, part of the above-chance overlap may reflect weights that are cheap to delete in this model regardless of task. A preflight on the published 7B masks found that a GRPO math mask trained on a different Qwen2.5-7B checkpoint overlaps the ToolAlpaca mask 7.0

Model	Precision	Base	Cartridge	Gained / lost	McNemar p	Deleted
Qwen3.5-9B, seed 1	bf16	42	69	27 / 0	1.5×10^{-8}	0.6645%
Qwen3.5-9B, seed 2	bf16	42	71	30 / 1	3.0×10^{-8}	0.6799%
Qwen3.5-9B, seed 3	bf16	42	68	26 / 0	3.0×10^{-8}	0.6718%
Qwen3.5-9B, seed 1	MLX 4-bit	38	66	28 / 0	7.5×10^{-9}	0.6645%
Qwen3.5-9B, seed 2	MLX 4-bit	38	66	28 / 0	7.5×10^{-9}	0.6799%
Qwen3.5-9B, seed 3	MLX 4-bit	38	61	24 / 1	1.5×10^{-6}	0.6718%
Qwen3.8-27B (run 1)	bf16	42	65	29 / 6	1.2×10^{-4}	0.5037%
Qwen3.8-27B (run 1)	MLX 4-bit	41	67	28 / 2	8.7×10^{-7}	0.5037%
LFM3-1.7B	bf16	11	59	48 / 0	7.1×10^{-15}	0.713%
Qwen2.5-7B (published)	MLX bf16	41	68	–	–	0.783%

Table 1: ToolAlpaca results (97 items). The published 7B mask was trained with SDFT; all other rows were trained by us with SFT. "Deleted" is the share of gate-projection weights the mask removes. 4-bit 9B rows are the nearest-zero form; requantize scores are in §5.3. The 4-bit 9B p values are exact two-sided McNemar tests on the listed discordant pairs. A second 400-step 27B run deleted 0.6049% and also scored 65; we use run 1 because it reaches the same score with fewer deletions (28,730,353 of 5,704,253,440 weights, 57.2 MB).

Pair (9B ToolAlpaca)	$\lvert A \cap B \rvert$	Expected Npq	J	J_0	J/J_0
seed 1 / seed 2	905,170	72,766	0.0436	0.00337	12.9
seed 1 / seed 3	909,807	71,900	0.0441	0.00335	13.2
seed 2 / seed 3	907,727	73,562	0.0435	0.00339	12.8
all three	777,898	489	–	–	1,591× (obs/exp)
9B recall seed 1 / seed 2	797,965	14,882	0.0887	–	53.6× (obs/exp)
7B ToolAlpaca / Science (published)	1,140,547	263,271	0.0241	0.00546	4.4

Table 2: Mask overlap. $N = 1{,}610{,}612{,}736$ for the 9B and 1,901,068,288 for the 7B. The 9B seed rows were recomputed from the mask files for this paper and match the values recorded at training time.

times above chance, which suggests such a task-independent component exists. We have not yet trained a matched random-task control for the 9B. Second, overlap is a statement about deleted sets; we have not shown that the core alone carries the gain.

5.3 Cartridges survive 4-bit quantization and toggle live

On mlx-community/Qwen3.5-9B-MLX-4bit, the 4-bit base scores 38 of 97, four below bf16. That base is identical item by item across the three seed evaluations. All three seeds keep a large gain under both application forms (Table 3):

For seed 1 the 4-bit gain (28 items) is slightly larger than the bf16 gain for the same mask (27). Nearest-zero is the form that can be toggled on a quantized model without touching kept weights, and it matches requantize here to within one item. On the 27B model, the run-1 cartridge on a 4-bit MLX build goes from 41 to 67 (28 gained, 2 lost, $p = 8.7 \times 10^{-7}$), against 42 to 65 in bf16.

Swap timings on the 4-bit 9B (nearest-zero form, all 32 gate projections, 10 repetitions): cold apply median 309 ms, warm apply 0.04 ms, eject 0.10 ms. After eject, a host-side byte comparison of every touched weight, scale and bias array matched the originals, and the logits equalled the base model's in 20 of 20 checks. We also saw lower decode throughput with the requantized form

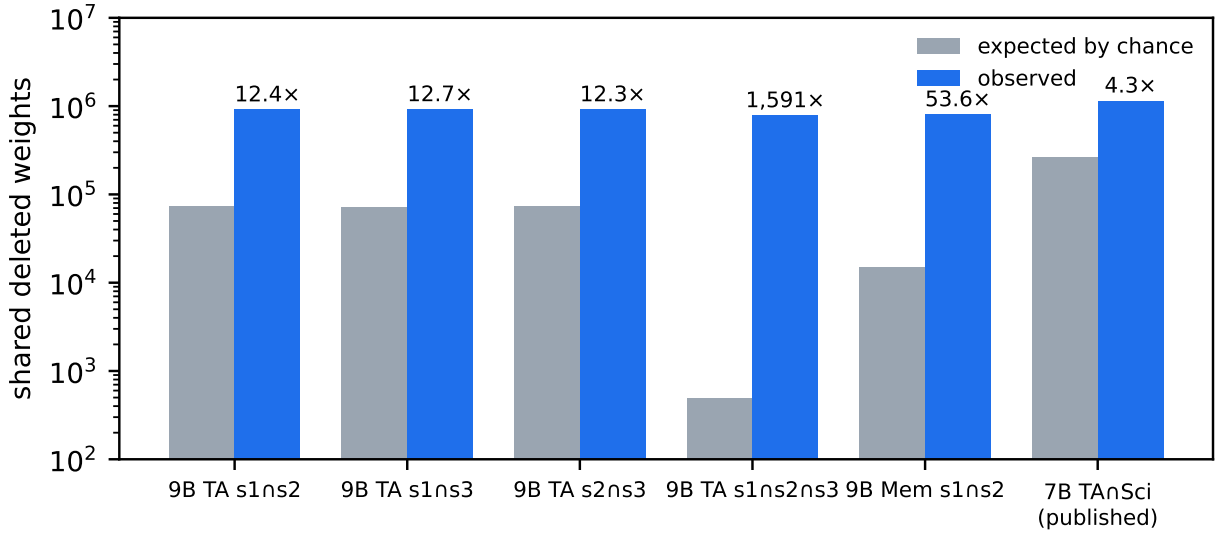


Figure 2: Observed shared deleted weights against the number expected if each mask deleted weights independently at its own density (log scale). Labels give observed/expected.

Seed	Nearest-zero	Gained / lost	Requantize	Gained / lost
1	66	28 / 0	66	28 / 0
2	66	28 / 0	65	27 / 0
3	61	24 / 1	62	25 / 1

Table 3: ToolAlpaca on the 4-bit 9B (97 items, shared base 38). Seed 3 lags the other two by a few items in both forms. Exact McNemar p is 7.5×10^{-9} for each 28/0 result, 1.5×10^{-8} for 27/0, 1.5×10^{-6} for 24/1 and 8.0×10^{-7} for 25/1.

in one batch-1 measurement (43.2 versus 65.6 tokens/s); a re-measurement was inconclusive, and we make no throughput claim.

5.4 Union stacking

Lu et al. suggest in their limitations section that masks for different tasks "might also be combined, for example by taking their union." We test that operator on two pairs: the published Qwen2.5-7B-Instruct masks [18], and two cartridges we trained on Qwen3.5-9B. Figure 4 shows the scores.

Published 7B masks. ToolAlpaca deletes 14,892,491 weights (0.783%) and Science 33,607,213 (1.768%). Their union deletes 47,359,157 weights (2.49%), which is 97.6% of the plain sum, so the two masks are close to disjoint ($J = 0.0241$, 4.4 times chance).

- ToolAlpaca: base 41, tool mask 68, science mask 49, union 64, intersection 44. The union keeps $(64 - 41)/(68 - 41) = 23/27 = 85\%$ of the tool gain.
- Science: base 155, science mask 344, tool mask 160, union 340, intersection 166. The union keeps $(340 - 155)/(344 - 155) = 185/189 = 97.9\%$ of the science gain.

The intersection (1,140,547 weights) alone moves ToolAlpaca by only 3 items and Science by 11, so the shared weights do not carry either skill by themselves. Each mask applied alone barely helps or hurts the other task (tool mask on Science: +5; science mask on ToolAlpaca: +8). Our MLX

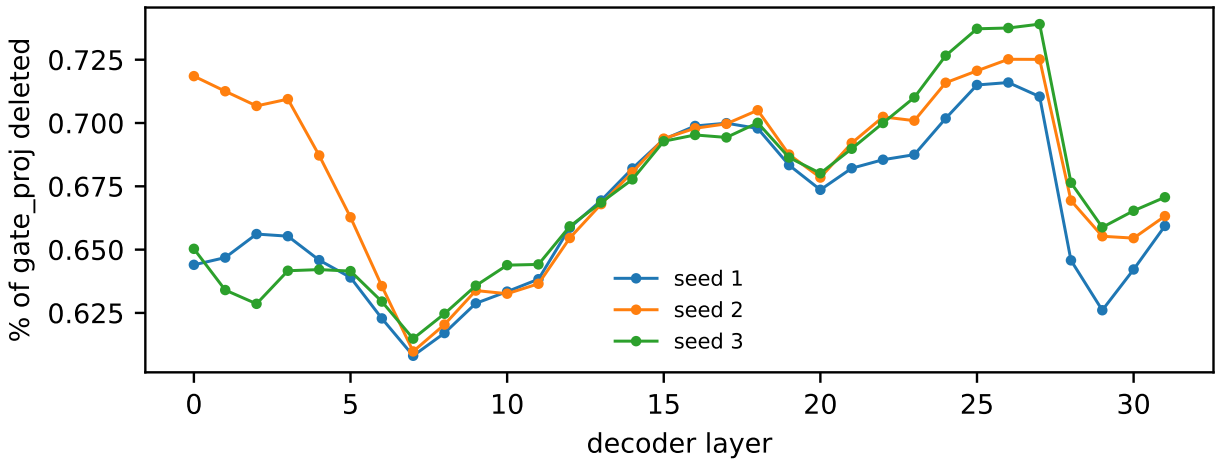


Figure 3: Share of gate-projection weights deleted per decoder layer, Qwen3.5-9B ToolAlpaca, three seeds.

numbers differ slightly from those reported in [18] for the same masks (ToolAlpaca 68 versus 67 of 97; Science 344 versus about 358 of 507), which we attribute to the different inference stack.

9B cartridges. We repeated the test with our seed-1 ToolAlpaca cartridge (10,702,823 deleted, 0.6645%) and a Science cartridge trained for 330 of the planned 400 steps (8,696,798 deleted, 0.5400%), scored with MLX in bf16. Their union deletes 16,530,526 weights (1.026%) and their intersection 2,869,095 ($J = 0.1736$, against a chance Jaccard of 0.00299, or 58 times chance). These two 9B masks overlap more than the published 7B pair; the union is 85.2% of the plain sum.

- ToolAlpaca: base 45, tool mask 69 (24 gained, 0 lost), science mask 49 (9 gained, 5 lost), union 66 (22 gained, 1 lost, $p = 5.7 \times 10^{-6}$), intersection 53. The union keeps $(66 - 45)/(69 - 45) = 21/24 = 87.5\%$ of the tool gain.
- Science (507 items): base 120, science mask 316 (211 gained, 15 lost, $p = 1.94 \times 10^{-45}$), tool mask 204 (113 gained, 29 lost), union 298 (199 gained, 21 lost, $p = 1.5 \times 10^{-37}$), intersection 257. The union keeps $(298 - 120)/(316 - 120) = 178/196 = 90.8\%$ of the science gain. Against the science mask alone the union is 18 items lower (46 gained, 64 lost), which is not significant ($p = 0.105$).

Unlike the 7B pair, the 9B tool mask alone already lifts Science substantially (120 to 204). The intersection also moves Science more than it does on the 7B (120 to 257, still well below the union and the solo mask). Both observations are consistent with the higher overlap, but we have not isolated a cause. The Science cartridge itself is a strong skill: 316 of 507, from a base of 120, even though training stopped early at 330 steps.

5.5 A second architecture: LFM3-1.7B

LFM3-1.7B combines gated short convolutions with sliding-window and global attention [16]; its MLP is gated, so the method applies to the gate matrix `w1` without changes. A ToolAlpaca cartridge (2,690,252 weights, 0.713%, 4.98 MB) lifts it from 11 to 59 of 97 (48 gained, 0 lost, $p = 7.1 \times 10^{-15}$) after a 290-second, $0.22 job. Much of this gain is format compliance: the median response shrinks from 1,152 to 205 characters, and the base model often fails exact match by adding text around a correct call. We count this as a real gain under the benchmark's rules but not as evidence of better

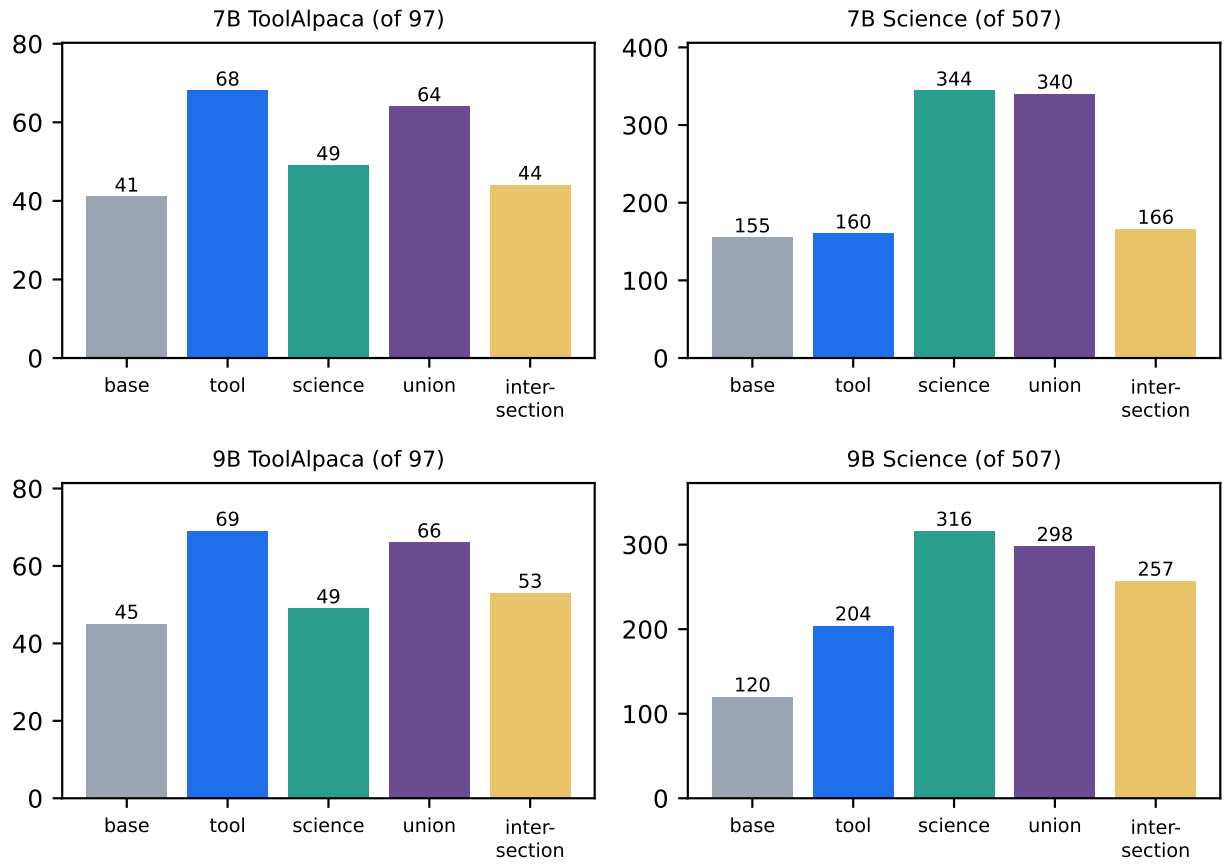


Figure 4: Union stacking. Top: the two published Qwen2.5-7B-Instruct masks of [18], trained with SDFT (MLX, bf16). Bottom: a ToolAlpaca cartridge and a Science cartridge we trained on Qwen3.5-9B (MLX, bf16). "union" deletes every weight that either mask deletes; "intersection" is the control that deletes only the shared weights.

tool reasoning.

5.6 A recall cartridge

On latest-value recall (Section 4) the 9B base gets all four values right on 89 of 200 test items. With the recall cartridge it gets 200 of 200 (111 gained, 0 lost, $p = 7.7 \times 10^{-34}$). On the longer held-out set it goes from 8 to 100 of 100 (92 gained, 0 lost, $p = 4.0 \times 10^{-28}$). A second seed replicated the result (90 to 200; 8 to 99), and the two recall masks (0.315% and 0.293% deleted) share 797,965 weights against 14,882 expected, 53.6 times chance. The cartridges are the smallest we trained, 10.8 and 10.1 MB.

This is a narrow, synthetic task with a fixed format, and the longer set changes only scale. The result shows that deletion alone can switch on a precise state-tracking behaviour that the base model does poorly and that it extends to inputs about five times longer than those seen in training. It does not show better long-context memory in general; standard long-context benchmarks are future work.

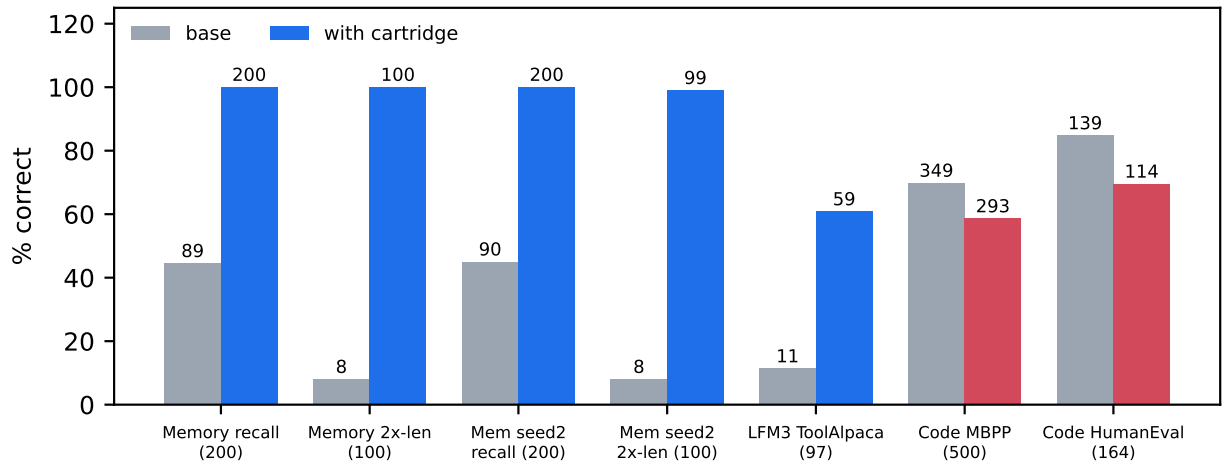


Figure 5: Recall, LFM3 and code cartridges. Bars are % correct; labels are counts. Red bars mark the code cartridge, which lowered accuracy.

5.7 A negative result: code

A cartridge trained on MBPP reference solutions (12,678,451 weights, 0.787%) made the 9B worse. MBPP test pass@1 fell from 349 to 293 of 500 (36 gained, 92 lost, $p = 7.9 \times 10^{-7}$) and HumanEval from 139 to 114 of 164 (7 gained, 32 lost, $p = 7.0 \times 10^{-5}$). Inspection of outputs shows the model imitating the short, unguarded style of the MBPP reference solutions, which are weaker than what the base model already writes. With SFT, a cartridge can only move the model toward its targets, so the targets have to be better than the model. We expect better-quality targets, self-distillation [25] or reward-based training (the setting where [18] report their largest gains) to behave differently, but have not tested them.

5.8 Forgetting, perplexity and KL on the 27B cartridge

We scored the run-1 27B ToolAlpaca cartridge on a 4-bit MLX build (mlx-community/Qwen3.8-27B-4bit, affine, group 64) against four off-task benchmarks, then WikiText-2 perplexity and forward KL, on an Apple-silicon Mac. An independent review re-derived the totals from the raw rows and matched them. The four tasks used the same seed-1234 item lists as a prior 0.5B run, but a 34.48% evenly spaced subset of each list (1,528 of 4,439 items), because a full pass was estimated at 5.8 hours. The same items were scored with the mask off and on. Log-likelihood scoring was used throughout.

All four tasks move down, each by less than 1.2 points, and the CI for the mean excludes zero. We read this as a small loss, about one point or less, that is probably real. Two caveats apply. First, the comparison rests on 26 flipped items. Second, log-likelihoods from this scorer can move by up to 1.0 nat when the same item is scored under a different batch shape (checked on 64 item-arms, all of which kept the same choice); both arms used identical batches, so the comparison is paired, but items with a narrow top-2 margin can flip for numerical reasons, and the CI covers item sampling only.

On the WikiText-2 raw test set (first 50,000 tokens, 48 windows of 1,024, 49,104 scored tokens, tokenized with the Qwen3.8 tokenizer) perplexity is 7.5956 with the mask off and 7.5712 with it on, a change of -0.0243 (95% CI -0.0378 to -0.0114): slightly better with the cartridge, not worse.

Task	n / full	Base	On	Diff (pts)	Lost / gained	p
HellaSwag (acc_norm)	344 / 1,000	81.69%	81.40%	−0.29	1 / 0	1.0
MMLU (acc)	344 / 1,000	80.81%	79.94%	−0.87	5 / 2	0.45
ARC-Challenge (acc)	404 / 1,172	60.40%	59.65%	−0.74	5 / 2	0.45
WinoGrande (acc)	436 / 1,267	78.44%	77.29%	−1.15	8 / 3	0.23
Mean of four tasks	1,528	75.33%	74.57%	−0.76	19 / 7	–

Table 4: Off-task scores for the 27B ToolAlpaca cartridge, 4-bit MLX. The mean difference has a paired bootstrap 95% CI of −1.41 to −0.13 points. No single task is significant. Pooled over the 1,528 items, 19 are lost and 7 are gained (exact McNemar $p = 0.029$).

Mean forward KL per token, KL(base ‖ on), is 0.00574 nats (window CI 0.00528 to 0.00624); the median, 95th percentile and maximum are 0.00257, 0.0171 and 4.57. Top-1 token agreement is 96.61%. With the cartridge ejected, KL is exactly 0 on all 49,104 tokens and the log-probabilities match the loaded base. Prompt and decode throughput and peak memory are not measurably different with the mask on (median prompt 63.5 versus 63.7 tokens/s; decode 11.64 versus 11.73; peak MLX memory 18.26 GB in both arms). This is one seed, one mask, one 4-bit scheme, and a 34% subset; we have not yet run the same suite on the 9B cartridges.

5.9 Cost

Hugging Face Jobs billed the RTX PRO 6000 at $2.75 per hour. A 400-step 9B ToolAlpaca run took 730 to 742 s inside the container (download, Wanda calibration, base evaluation, training at 1.05 to 1.08 s per step, and the masked evaluation), about $0.60 billed. Measured job costs were $0.96 and $0.98 for the two recall runs, $0.75 for code and $0.22 for LFM3. The first 27B run took 26 minutes on two GPUs for $2.38; the cartridge is 57.2 MB compressed (28,730,353 of 5,704,253,440 gate-projection weights, 0.5037%). A 9B cartridge is 20.2 to 20.5 MB compressed, about 0.1 bits per masked weight and roughly 1/900 of the 19.3 GB bf16 checkpoint.

6 Related work

Deletion Tuning and mask learning. Our work is a direct extension of Deletion Tuning [18], which itself relates to MaskLLM [9], L_0 relaxations [17] and Wanda [26]. Learning binary masks over frozen weights to add tasks goes back at least to Piggyback [20] and PackNet [19]; supermasks showed that masks alone can define a working network [24], and Supermasks in Superposition stored many task masks over one fixed network and selected among them [29]. The lottery ticket hypothesis [11] is the classical background. Our contribution is to test this per-task mask idea on current instruction-tuned LLMs with deletion-only masks, under quantization and composition, on small hardware.

Sparse and parameter-efficient fine-tuning. Diff pruning [12] and sparse fine-tuning [1, 2] learn sparse value updates. Ansell et al. already composed sparse fine-tunings by addition for cross-lingual transfer [1], a close precedent for our union operator. SHiRA [4] trains a small fraction of weights directly to enable fast adapter switching and multi-adapter fusion on device, which is the deployment goal we share; unlike SHiRA, a cartridge writes no new values and stores one bit per masked weight. LoRA [14] and QLoRA [7] are the standard alternatives. We have not yet run a LoRA baseline on the same data and steps.

Model merging and task localization. Task arithmetic [15], TIES-Merging [31], DARE [32] and model soups [30] combine fine-tuned weights. TALL-masks [28] learn binary masks that localize each task's information inside a merged task vector, and show that tasks occupy largely separate weights; our near-disjoint 7B masks are consistent with that picture. Knowledge-neuron and ROME-style editing [6, 22] localize facts in MLP weights; we plan to label the neurons that cartridges delete in the same spirit.

Name collision. "Cartridges" is also the name of a method by Eyuboglu et al. [8] that trains small KV-cache representations of a long corpus through self-study. That work is unrelated in mechanism: it stores learned key–value states, while a skill cartridge is a weight-deletion mask. We use "skill cartridge" throughout to keep the two apart.

7 Limitations

- **Small test sets.** ToolAlpaca has 97 items. Per-model gains are highly significant, but between-model comparisons (9B versus 27B) are not tested for significance and differ by about 4 items.
- **Confounded size comparison.** Qwen3.5-9B and the 27B model are different generations.
- **Mostly in-distribution evaluation.** ToolAlpaca, Science and recall test the trained format. We have no broad out-of-distribution suite yet. Forgetting, perplexity and KL are reported for the 27B ToolAlpaca cartridge on a 34% subset of four tasks in 4-bit (§5.8); the 9B cartridges have not been through the same suite.
- **Format effects.** Part of every exact-match gain may be formatting. For LFM3 this is clearly a large part.
- **Stacking was tested with two skills, on one 7B pair and one 9B pair.** Union kept 85% and 87.5% of the tool gain and 97.9% and 90.8% of the science gain, so interference is not zero. The 18-item 9B science drop from solo to union is not significant ($p = 0.105$). More than two cartridges are untested.
- **The recall task is synthetic.**
- **Code failed** with SFT on reference solutions; we do not know yet whether any deletion-only recipe works for code at this scale.
- **No LoRA or full fine-tuning baseline** on our own tasks and models. [18] provide those comparisons for their settings; we have not repeated them.
- **4-bit results cover one quantization scheme** (MLX affine, 4 bits, group 64) and two models. On the 9B they cover three seeds and two application forms. GGUF and other schemes are untested.
- **Single evaluation per condition** under greedy decoding; we did not sample.
- **Model naming.** The 27B checkpoint is the Hugging Face model `Qwen/Qwen3.8-27B`, as recorded in the training job; the 4-bit build is `mlx-community/Qwen3.8-27B-4bit`.

8 Ongoing work

Neuron-level labels for the weights each cartridge deletes (using the Goodfire neuron atlas) are in progress and are not reported here. We next plan to train cartridges, including a code cartridge, with on-policy self-distillation (SDFT), the recipe of the published 7B masks; we have no results from it yet.

9 Conclusion

Deletion Tuning masks work as skill cartridges on the kind of hardware a single person owns. A 9B model with a 21 MB ToolAlpaca cartridge matched a 27B model with its own cartridge, the result held across three seeds with a shared core of deleted weights, the gain survived 4-bit quantization on all three seeds with a sub-millisecond live toggle, and two published 7B masks and two 9B masks stacked by union while keeping most of their gains (85% and 87.5% of the tool-use gain; 97.9% and 90.8% of the science gain). The method carried over to a hybrid convolution and attention architecture and to a recall skill. Code did not work with our recipe. On a 4-bit 27B build, four general-knowledge tasks dropped by a mean of 0.76 points. All of this rests on the method of Lu et al. [18], including the union operator they proposed; what we add is evidence that its masks can be treated as small, reproducible, composable files and served live from a quantized base.

Code and data availability

The training scripts, the MLX loader (live apply and eject, bf16 and 4-bit), the evaluation and statistics code, and this paper's source and figures are at `https://github.com/austindixson/skill-cartridges` (MIT license). The cartridges reported here (three 9B ToolAlpaca seeds, the 9B Science cartridge, two 9B recall seeds, the 27B run-1 ToolAlpaca cartridge and the LFM3-1.7B ToolAlpaca cartridge) are at `https://huggingface.co/austindixson/skill-cartridges`, with each one's training summary and per-step metrics. The negative-result code cartridge is not distributed. ToolAlpaca and Science data and scorers are not redistributed; they are fetched from the SDFT authors' repository at the pinned commit given in Appendix A.

Acknowledgments. We thank Jack Lu, Zhenbang Yang, Mike Lasby, Tejas Pote, Yani Ioannou and Mengye Ren for releasing the Deletion Tuning code and masks, and Idan Shenfeld and coauthors for releasing the ToolAlpaca and Science data and evaluators. AI coding and writing assistants were used for experiment code, analysis and drafting; the author designed the study and verified all results.

References

[1] Alan Ansell, Edoardo Maria Ponti, Anna Korhonen, and Ivan Vulić. Composable sparse fine-tuning for cross-lingual transfer. *arXiv preprint arXiv:2110.07560*, 2021.

[2] Alan Ansell, Ivan Vulić, Hannah Sterz, Anna Korhonen, and Edoardo M. Ponti. Scaling sparse fine-tuning to large language models. *arXiv preprint arXiv:2401.16405*, 2024.

[3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[4] Kartikeya Bhardwaj, Nilesh Prasad Pandey, Sweta Priyadarshi, Viswanath Ganapathy, Shreya Kadambi, Rafael Esteves, Shubhankar Borse, Paul Whatmough, Risheek Garrepalli, Mart Van Baalen, Harris Teague, and Markus Nagel. Sparse high rank adapters. *arXiv preprint arXiv:2406.13175*, 2024.

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[6] Damai Dai, Li Dong, Yaru Hao, Zhifang Sui, Baobao Chang, and Furu Wei. Knowledge neurons in pretrained transformers. *arXiv preprint arXiv:2104.08696*, 2021.

[7] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *arXiv preprint arXiv:2305.14314*, 2023.

[8] Sabri Eyuboglu, Ryan Ehrlich, Simran Arora, Neel Guha, Dylan Zinsley, Emily Liu, Will Tennien, Atri Rudra, James Zou, Azalia Mirhoseini, and Christopher Ré. Cartridges: Lightweight and general-purpose long context representations via self-study. *arXiv preprint arXiv:2506.06266*, 2025.

[9] Gongfan Fang, Hongxu Yin, Saurav Muralidharan, Greg Heinrich, Jeff Pool, Jan Kautz, Pavlo Molchanov, and Xinchao Wang. MaskLLM: Learnable semi-structured sparsity for large language models. *arXiv preprint arXiv:2409.17481*, 2024.

[10] Kehua Feng, Xinyi Shen, Weijie Wang, Xiang Zhuang, Yuqi Tang, Qiang Zhang, and Keyan Ding. SciKnowEval: Evaluating multi-level scientific knowledge of large language models. *arXiv preprint arXiv:2406.09098*, 2024.

[11] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *arXiv preprint arXiv:1803.03635*, 2018.

[12] Demi Guo, Alexander M. Rush, and Yoon Kim. Parameter-efficient transfer learning with diff pruning. *arXiv preprint arXiv:2012.07463*, 2020.

[13] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on Apple silicon. https://github.com/ml-explore/mlx, 2023.

[14] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[15] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

[16] Liquid AI. Introducing LFM3: Every modality for specialized intelligence. Liquid AI Blog, https://www.liquid.ai/blog/lfm3, 2026.

[17] Christos Louizos, Max Welling, and Diederik P. Kingma. Learning sparse neural networks through L_0 regularization. *arXiv preprint arXiv:1712.01312*, 2017.

[18] Jack Lu, Zhenbang Yang, Mike Lasby, Tejas Pote, Yani Ioannou, and Mengye Ren. The surprising effectiveness of deleting weights in LLM post-training. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. arXiv:2611.10866. Code: https://github.com/agentic-learning-ai-lab/deleting-weights; masks: https://huggingface.co/Jacklu0831/deletion-tuning-masks.

[19] Arun Mallya and Svetlana Lazebnik. PackNet: Adding multiple tasks to a single network by iterative pruning. *arXiv preprint arXiv:1711.05769*, 2017.

[20] Arun Mallya, Dillon Davis, and Svetlana Lazebnik. Piggyback: Adapting a single network to multiple tasks by learning to mask weights. *arXiv preprint arXiv:1801.06519*, 2018.

[21] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[22] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. *arXiv preprint arXiv:2202.05262*, 2022.

[23] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[24] Vivek Ramanujan, Mitchell Wortsman, Aniruddha Kembhavi, Ali Farhadi, and Mohammad Rastegari. What's hidden in a randomly weighted neural network? *arXiv preprint arXiv:1911.13299*, 2019.

[25] Idan Shenfeld, Mehul Damani, Jonas Hübotter, and Pulkit Agrawal. Self-distillation enables continual learning. *arXiv preprint arXiv:2601.19897*, 2026.

[26] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695*, 2023.

[27] Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. ToolAlpaca: Generalized tool learning for language models with 3000 simulated cases. *arXiv preprint arXiv:2306.05301*, 2023.

[28] Ke Wang, Nikolaos Dimitriadis, Guillermo Ortiz-Jimenez, François Fleuret, and Pascal Frossard. Localizing task information for improved model merging and compression. *arXiv preprint arXiv:2405.07813*, 2024.

[29] Mitchell Wortsman, Vivek Ramanujan, Rosanne Liu, Aniruddha Kembhavi, Mohammad Rastegari, Jason Yosinski, and Ali Farhadi. Supermasks in superposition. *arXiv preprint arXiv:2006.14769*, 2020.

[30] Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. *arXiv preprint arXiv:2203.05482*, 2022.

[31] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-merging: Resolving interference when merging models. *arXiv preprint arXiv:2306.01708*, 2023.

[32] Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. *arXiv preprint arXiv:2311.03099*, 2023.

A Reproducibility details

Code. Upstream `agentic-learning-ai-lab/deleting-weights` at commit `94f0340ebb574f95bb41d844165253a6785e215e`; SDFT data and scorers from `idanshen/Self-Distillation` at commit `d77573212fa0a3ae2eeb64b9b44db1c251f75e3e`. Training environment: torch 2.15.0 (CUDA 13.2 build), transformers 5.22.2, accelerate 1.15.0, datasets 5.2.0, safetensors 0.8.0, zstandard 0.25.0, flash-linear-attention 0.6.0. Inference on Apple silicon: MLX 0.32.4, mlx-lm 0.32.2. The training scripts, loader and cartridges are public at https://github.com/austindixson/skill-cartridges and https://huggingface.co/austindixson/skill-cartridges.

Hyperparameters. Table 5.

Setting	Value
Target	every text-layer MLP gate projection (`gate_proj`; `w1` for LFM3)
Steps / batch	400 / 4 (27B run 1: 272 steps on 2 GPUs)
Optimizer	AdamW (fused), lr 5×10^{-4}, weight decay 0, clip 1.0
Sharpness α	linear $100 \to 1000$
Init	Wanda, $\eta = 1/2048$, prior strength 3, 256 calibration prompts
Regularizer	$\lambda = 10^{-5}\,(1 - t/T)$
Loss	mean cross-entropy over answer tokens
Max length	2048 (recall: 3072)
Seeds	ToolAlpaca 0, 2, 3; recall 0, 2; others 0
Decoding	greedy, chat template, thinking off
Max new tokens	ToolAlpaca 1024, Science 2048, recall 64
Hardware	1× RTX PRO 6000 96 GB (HF Jobs); MLX on Apple silicon (4-bit, stacking)

Table 5: Training and evaluation settings.

Overlap computation. Masks are decoded from the upstream format; for every mask the decoded deleted count equals `zero_params` in its config. Per-layer padding bits are excluded. Intersections are exact popcounts.

Per-run measurements (9B ToolAlpaca). Seconds per step 1.050 / 1.057 / 1.080; training time 480 / 482 / 492 s; in-container wall time 742 / 730 / 742 s; final training cross-entropy 0.129 / 0.124 / 0.061; compressed mask 21,141,031 / 21,521,657 / 21,322,108 bytes; peak GPU memory 49.1 GB.